

SmartHome Control

Introduction

Notre entreprise a été choisie par une grande figure du domaine de la domotique afin de concevoir et développer un système d'automatisation pour maisons connectées.

Le système doit permettre aux utilisateurs de gérer et contrôler leurs équipement domestique et d'automatiser certaines tâches à partir d'une interface simple et accessible.

Le projet présente des enjeux importants :

- Sécurité des données et équipements
- Fiabilité des commandes
- Disponibilité du système
- Simplicité d'utilisation
- Évolutivité et maintenance
- Traçabilité des opérations
- Respect des exigences en matière de données personnelles

Après analyse du contexte, le Cycle en V est retenu comme méthodologie de développement.

Objectifs du Projet

SmartHome Control s'articule autour de ces fonctionnalités clés :

- Gestion des utilisateurs consultation de leur statuts
- Gestion de un ou plusieurs foyer et consultation de leur état
- Gestion des équipements consultation de leur état
- Contrôle des équipement à distance
- Planification des tâches
- Recevoir des notifications
- Consulter un historique des actions effectués
- Gestions des dysfonctionnement et des erreurs

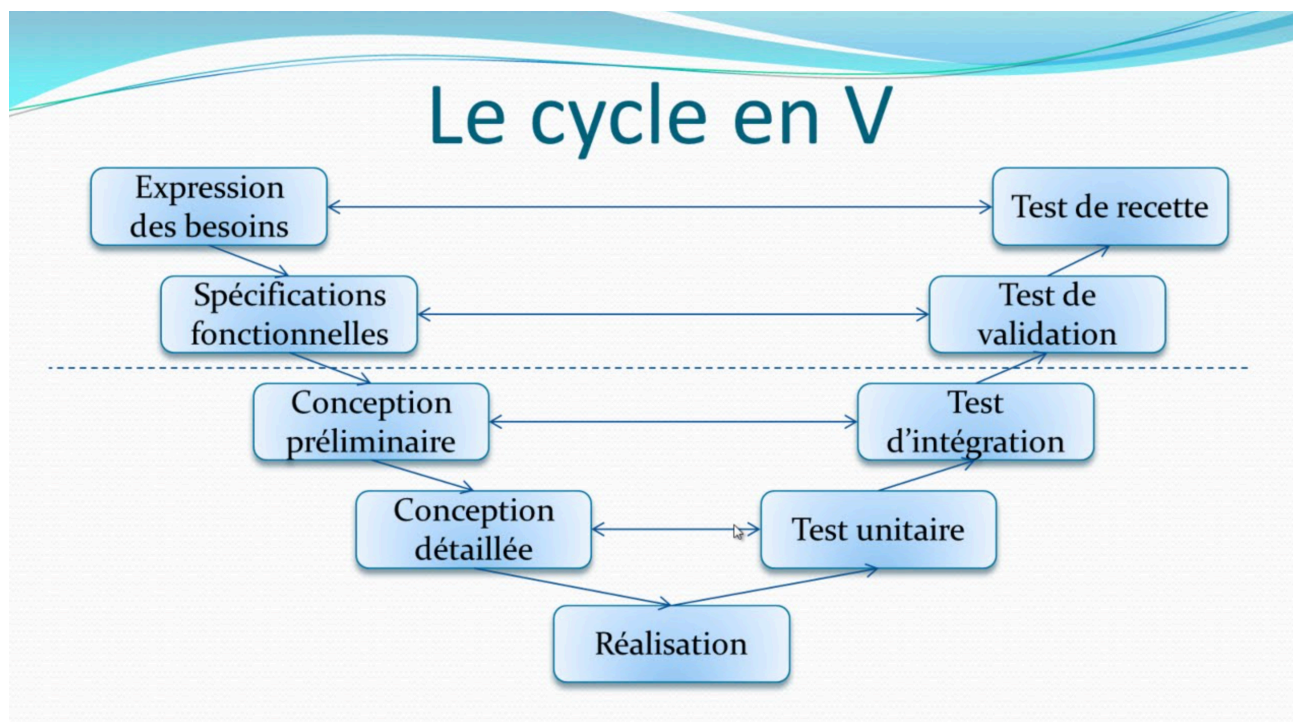
Un utilisateur peu créer une règle qui entraîne une action :

Si la température descend en dessous de 19°C entre 20h et 06h, allumer le chauffage à 21°C.

Le système devra alors gérer :

condition —> déclencheur —> action —> confirmation —> notification —> historique

Pourquoi choisir le Cycle en V ?



Le Cycle en V, en gestion de projet, est une méthodologie de travail qui découle du modèle Waterfall permettant de représenter des processus de développement de manière linéaire et en phases successives. Mais il s'agit avant tout d'ingénierie système. Ce modèle a été développé dans les années 1980 et appliqué au champs des projets industriels puis étendu aux projets informatiques. Le cycle en V apporte une réponse au problème des tests improvisés et livrables qui ne répondent plus au cahier des charges initial et aux retards évitables.

Il est particulièrement efficace sur les projets dont le cahier des charges inchangé du début à la fin, dont l'environnement technique évolue très peu, dont la documentation est exhaustive à chaque phase et pour des projets simple avec des objectifs clairs et des délais courts.

La lettre V fait référence à la vision schématique de ce cycle qui se constitue d'une phase descendante puis d'une phase ascendante. Il associe à chaque phase de réalisation une phase de validation. Chaque phase descendante trouve son miroir exact côté validation. Donc la phase de gauche définit ce que le système doit faire et comment il sera construit et la phase de droite vérifie que le système construit correspond bien à ce qui avait été défini.

Nous avons donc trois grands axes :

Phase descendante	=	Conception
Pointe du V	=	Mise en oeuvre et développement
Phase ascendante	=	Vérification validation

Phase descendante :

La phase descendante concerne tout ce qui est lié à la conception du projet. Il s'agit de la phase de conception pendant laquelle les besoins sont définis grossièrement et détaillés de plus en plus précisément.

Analyse et expression du besoin :

La première étape de cette phase est l'exigence ou les expressions du besoins. Il est nécessaire et même crucial que la structure et les exigences du projet soit claires et sans ambiguïté. Cette étape doit porter sur des éléments nécessaires, rédigé dans un langage simple à lire, cohérents, réunis au même endroit, précis et incluant une marge d'erreur, accessibles, et vérifiables via une inspection, une analyse une démonstration ou un test. C'est ici que l'on va rassembler le plus d'informations possible. Les parties suivantes vont dépendre de cette étape cruciale. On va donc y investir beaucoup de temps. Nous allons y retrouver l'objectif du projet, les besoins clients, le public cible, le MVP, les contraintes fonctionnelles, les moyens techniques, les délais, le budget et le plan de management du projet (organisation de l'équipe), la validation des tests et quels sont les risques et comment les gérer ainsi que le plan de recette (critères d'acceptation). Cette étape est sous la responsabilité du maître d'ouvrage.

Les spécifications fonctionnelles :

Ici, on entre dans la partie analyse des spécifications fonctionnelles du projet. On obtiendra donc le cahier des charges fonctionnel qui contiendra les règles de gestion métier, les users stories, les wireframes et maquettes et la planification des tâches et le plan de validation et éventuellement des diagrammes de cas d'utilisation, de séquence et le modèle de conception de base de données (MCD). Une stratégie de tests fonctionnels ou end-to-end (en fonction des moyens et des délais) sera rédigé. Cette étape est sous la responsabilité du maître d'œuvre.

Conception générale :

Dans cette partie nous allons aborder les points suivants. L'infrastructure va définir le matériel utilisé et les pratiques nécessaires pour stocker et mettre en ligne l'application et ses données, notamment avec un diagramme de déploiement. Puis nous nous intéresserons aux choix technologiques pour définir quels langages et packages seront choisis pour coder l'application. La suite concerne les schémas de modèle logique de bases de données (MLD) qui apportera plus de précision sur comment les données de l'application seront organisées et liées entre elles. Ensuite, on a l'architecture logicielle. Nous expliquerons comment les différentes parties de l'application seront organisées et reliées entre elles. Enfin, on termine cette partie par les contraintes techniques qui servira à déterminer les niveaux d'exigence au sujet de la sécurité, la compatibilité ou des performances de l'application et l'ensemble des échanges entre le système et les services externes. Un plan de tests d'intégration sera rédigé à cette étape.

Conception détaillée :

La quatrième et dernière étape de la phase descendante est la conception détaillée avant la phase de développement de l'application. On spécifiera davantage la structure interne de l'application et donc de chaque module, chaque composant, la gestion des exceptions et la structure des classes et fonctions. Pour cela on pourra réaliser des diagrammes de composant, des diagrammes d'état, et des diagrammes de classe. Nous entrerons ensuite en détails dans la structure de la base de données avec le modèle physique de la base de données (MPD) qui contiendra les tables, les champs, les relations, les types, et les contraintes. On rédigera les tickets utilisés en phase de développement. Un plan de tests unitaire sera rédigé pour s'assurer du bon fonctionnement individuel de toutes les parties de l'application. On détermine aussi comment la maintenance et la mise à jour est définie.

Point important :

Il est très important que le client valide l'expression du besoin et la partie spécification technique.

Dans le cas où cette validation n'est pas strictement effectuée, on prends le risque de partir sur de mauvaises bases. À cause de cela, on s'expose à des failles et des risques de reprise structurelles qui ne sont pas adaptés au cycle en V mais plus aux méthodes agiles.

Phase de développement (mise en œuvre) :

La pointe du V représente la phase de développement du projet. Les développeurs vont prendre les tickets, qui sont basés sur les phases précédentes et les exécuter.

Toute l'application, front-end et back-end, est développée dans cette étape. Les retours clients sont donc bien moins fréquents que dans la méthode agile, d'où l'intérêt d'être exhaustif et précis lors de la phase descendante.

Phase ascendante :

Cette branche de droite regroupe toutes les étapes de validations et de tests définies dans la phase de gauche et se conclue par la mise en ligne de l'application.

Tests unitaires :

En se basant sur le plan de tests unitaires, on va exécuter les tests unitaires de l'application. L'objectif est de vérifier chaque sous composant et fonction de l'application de manière isolée.

Tests d'intégration :

On vérifie que les différentes parties de l'application communiquent correctement entre elles et que le système communique avec les services externes de l'application, le tout sans créer de conflit.

Tests systèmes (ou validation) :

Les tests fonctionnels ou end-to-end sont exécutés par le maître d'œuvre afin de vérifier que le système complet fonctionne et respecte les spécificités techniques et fonctionnelles. On lance aussi les tests de sécurité, de performance, de charge, de récupération et de non régression.

Tests de recette (ou acceptation) :

Validation finale par le client ou le maître d'ouvrage pour confirmer que le besoin exprimé au début du projet est rempli.

Mise en service :

Une fois toutes les étapes passées, le projet est déployé.

Avantages :

Méthode simple à mettre en œuvre si les besoins et exigences sont clairement définis
Besoins exprimés avant le dev, donc limite les interprétations différentes entre client, chef de projet, développeurs, UX/UI, QA, architectes, autres.
Objectifs clairs et tests de contrôle à chaque phase
Phase de test garanti un produit final avec le moins d'erreurs possibles voir aucune
Validation anticipée : les tests sont réfléchis dès la conception. TDD possible. Détecter les incohérences avant le dev.
Traçabilité chaque exigence associée à architecture/composant, fonction, tests

Erreurs courantes et limites de la méthode :

- Sous estimer la phase d'analyse des besoins (cahier des charges incomplet = reprise en cascade, spécification fonctionnelles floues)
- Ignorer l'**effet tunnel** : pas de retour avec les parties prenantes entre conception et tests. Risque produit conforme aux spécifications mais pas aux attentes réelles du client
- Application aux exigences changeantes : Si plus de 20% des exigences ont été modifiées entre la mi-projet ce n'est plus la bonne méthode.
- Valider une phase sous pression de planning dans réelle vérification
- Détection tardive des problèmes
- Inadapté aux projets longs et complexes et modulables

Palier aux erreurs et inconvénients :

- utiliser des approches et itérations courtes pour des cycles rapides et avoir un retour rapidement = livrables sous forme de version
- Avec la présence d'un prestataire qui est impliqué l'ensemble des étapes

Pourquoi le cycle en V est adapté au Projet SmartHome :

Sécurité

Interaction avec des équipement physiques

Besoin de traçabilité

Idées de fonctionnalités optionnelles :

Radio/music/enceintes (+ suivi dans les différentes pièces)

Usage de la voix = assistant local (enceinte intelligente (ou peut-être IA)

Surveillance et monitoring des températures et de la consommation électrique

Gestions des lumières

Désactivation des appareils consommant de l'électricité

Système d'alarme anti-effraction

Volets roulant automatiques (heure ou lever/coucher du soleil)

Gestion et visualisation des caméras de surveillance

Gestion du chauffage

Gestion de la climatisation

Détection de la présence dans le logement (proprio de l'appartement et application ou détecteur de mouvements = problème avec animaux de compagnie)

Sonnete avec caméra et micro => notification et visuel sur l'application

Détecteurs de fumée (alarme sonore)

Ventilation contre l'humidité (taux élevé)

Rappel de sorti des poubelles

Aspirateur automatique

Cafetiere (associé au réveil)

Budget :

Ex 1 :

Avec 100 à 250 €

- prises connectées ;
- quelques ampoules ;
- premier assistant vocal.

Avec 300 à 800 €

- thermostat connecté ;
- caméra ;
- éclairage de plusieurs pièces.

Avec 1 000 € et plus

- volets motorisés ;
- alarme complète ;
- serrure connectée ;
- pilotage global.

Ex 2 :

Niveau d'équipement :

Entrée de gamme	50 à 150€
Intermédiaire	3 000 à 10 000€
Complet	8 000 à 20 000€
Haut de game	+20 000€

Box domestique :

Entrée de gamme	50 - 150€
Milieu de gamme	150 - 500€
Haut de gamme	500 - 1500€

Éclairage connecté

Le prix va dépendre de la qualité et des fonctionnalités de l'élément :

Ampoule et interrupteur connecté	10 - 70€
Interrupteur connecté	80 - 150€
Détecteur de présence :	20 - 80€
Variateur connecté	40 - 120€

Chauffage, ventilation et climatisation

Sécurité et accès